

11. Transformer 模块 (Transformer Block)

本章目标

第 10 章讲完了注意力机制 (Attention)：每个词元 (Token) 能够跨位置读取、聚合别的词元信息。但注意力机制只完成了“信息交流”这一半，Transformer 真正的最小可堆叠单元还差几个零件：让每个词元独立“消化”信息的前馈网络 (FFN)、防止深层网络退化的残差连接 (Residual Connection)、稳定数值尺度的层归一化 (Layer Normalization)、抑制过拟合的随机失活 (Dropout)。本章会依次讲清这四个零件为什么必须存在，最后把它们组装成一个完整的 Transformer 模块 (Transformer Block)，并顺带接入位置信息与因果掩码 (Causal Mask)，为 11.1 节的小型 GPT (Mini GPT) 实践做好准备。

一、为什么只有 Attention 还不够？

第 10 章的 Attention 解决的是 Token 之间的信息交换：每个位置根据相关性，从其他位置聚合信息。但聚合完成后，每个 Token 还需要对自己的表示做一次非线性变换，才能形成新的特征组合。这个任务由 前馈网络 (Feed Forward Network, FFN) 完成。

二者分工明确：

- Attention：跨 Token 读取和聚合信息；
- FFN：对每个 Token 的向量独立进行非线性加工。

FFN 对所有位置使用同一组参数，但各位置分别计算，因此也叫 Position-wise Feed Forward Network。这和第 9 章 RNN 的 Parameter Sharing 是同一类思想：不管当前是序列里第几个 Token，都用同一套权重处理——只是 RNN 共享的是“跨时间步”，FFN 共享的是“跨序列位置”。

二、Feed Forward Network (FFN)

FFN 本质上是第 4 章 MLP 的直接应用：输入表示 $\rightarrow$ Linear $\rightarrow$ 激活函数 $\rightarrow$ Linear $\rightarrow$ 新表示。注意这里的“输入表示”是 Attention 输出、再经过残差连接后的 Token 向量，而不是第 3 章的原始 Embedding。它通常先升维，再经过非线性激活，最后降回原维度。

示例：一个 Token 的加工过程。

```
import torchx = torch.randn(4) # 一个 Token 的向量linear1 = torch.nn.Linear(4, 8)hidden = linear1(x)print(hidden.shape) # torch.Size([8])
```

Transformer 首先把维度扩大（例如 $4 \rightarrow 8$ ）。为什么？因为更大的空间能够表示更复杂的组合关系——如果一直停留在四维这样的低维空间，模型能表达的关系非常有限；扩展到八维，模型就可以自动组合出“颜色 $\times$ 速度”、“颜色 $\times$ 时间”等更多隐藏特征。因此 FFN 第一步通常是升维。

仅仅 Linear 还不够，因为两个 Linear 连续计算仍然等价于一个更大的 Linear，无法学习复杂关系。因此中间必须加入激活函数：Linear $\rightarrow$ GELU $\rightarrow$ Linear。现代 Transformer 大多数使用 GELU，而不是 ReLU。

每个 Token 的向量例如是 768 维 (GPT-2 Small)，Transformer 希望经过 FFN 后输出仍然保持 768 维，这样下一层才能继续处理。因此 FFN 通常采用“先扩展、再压缩”：768 $\rightarrow$ 3072 $\rightarrow$ 768。

示例：一个真正的 FFN。

```
import torch.nn as nnffn = nn.Sequential( nn.Linear(768, 3072), nn.GELU(), nn.Linear(3072, 768))
```

整个 FFN 只有两个 Linear 层（中间夹一个激活函数），却承担了 Transformer 一半以上的计算量。

FFN 为什么占用大量参数和计算？对标准 Transformer，Attention 的投影通常包含 Q、K、V、O 四个 $d_{\text{model}} \times d_{\text{model}}$ 矩阵；采用 $d_{\text{ff}} \approx 4d_{\text{model}}$ 的普通 FFN，则有两个约为 $d_{\text{model}} \times 4d_{\text{model}}$ 的矩阵。忽略偏置时，FFN 参数量约为 $8d_{\text{model}}^2$ ，Attention 投影约为 $4d_{\text{model}}^2$ ，因此 FFN 往往是参数和计算的重要来源。具体比例会随 SwiGLU、GQA、MoE 等结构变化，不能只用一个固定百分比概括。

Attention 负责建立关系，FFN 负责学习复杂映射，二者缺一不可：删除 Attention，模型失去上下文能力；删除 FFN，模型表达能力大幅下降。Transformer 实际上是在不断重复“交流 $\rightarrow$ 思考 $\rightarrow$ 交流 $\rightarrow$ 思考”，直到完成整个推理。

本节小结：[OK] FFN 本质上就是一个 MLP [OK] 每个 Token 独立经过同一个 FFN [OK] FFN 的结构通常是：Linear $\rightarrow$ GELU $\rightarrow$ Linear [OK] FFN 采用“升维 $\rightarrow$ 非线性 $\rightarrow$ 降维”的设计 [OK] FFN 负责信息加工，而不是信息交流

三、为什么需要 Residual Connection (残差连接) ？

Transformer Block 已经拥有 Attention $\rightarrow$ FFN，看起来可以不断堆叠：Attention $\rightarrow$ FFN $\rightarrow$ Attention $\rightarrow$ FFN $\rightarrow$ 。是不是层数越多，模型越聪明？答案并没有那么简单。

假设你已经训练好一个 Transformer，老板说“再加 20 层”，新模型一定更好吗？很多人会说当然，因为层数更多表达能力更强。但真正训练时，结果经常相反——模型不仅没有更好，反而更难训练，甚至准确率下降。为什么？

一个类比：100 个人传话，第一位说“今天下午两点开会”，传到最后可能变成“今天晚上开会”。信息在不断传递过程中会慢慢发生变化。神经网络也是一样： $x \rightarrow F_1(x) \rightarrow F_2(F_1(x)) \rightarrow F_3(\dots) \rightarrow \dots$ ，如果有 100 层，最开始的信息还能保留下来吗？越来越困难。

一个更严重的问题：梯度消失。训练需要反向传播，梯度必须从最后一层一路传回第一层。如果每一层都乘上一个较小的数字（例如 0.9），100 层以后梯度几乎变成 0，第一层几乎收不到任何更新——这就是 Vanishing Gradient（梯度消失）。

一个大胆的想法：修一条高速公路。既然信息容易丢失，为什么不给它增加一条捷径？

在往下看图之前，先把两个最容易看晕的符号说清楚——它们其实都不是什么新概念：

- x ：就是“进入这一步之前的数据”。可以简单理解成一个 Token 当前手里拿着的那份“稿子”。
- $+$ ：就是加法本身，图上专门画一个方框，只是为了让你看清楚“两份东西是在这里被加在一起的”，它不代表任何新的计算模块。

也就是说，一个 Token 的稿子会走两条路：一条送去给 Attention 修改，另一条原封不动直接送到终点；到了终点，这两份稿子会加在一起，谁也不会被扔掉。图里的虚线，画的就是“原封不动”这条路：

残差连接：实线走 Attention，虚线直接跳过

直接跳过



生成脚本见 [residual_shortcut_diagram.py](#)。

图里三个元素对应关系非常直接：x 是稿子本身；Attention 是“修改意见”；+ 是“把稿子和修改意见叠在一起”，而不是“用修改意见替换掉稿子”。这样原始信息可以直接传到最后，不会丢失。

示例：用具体数字走一遍这张图。

```
x = 10 # 图里的 x：进来的稿子，这里就取一个具体数字方便计算 f = x * 0.9 # 图里的 Attention：对稿子做了一点“修改”，算出结果 9 print(f) # 9 —— 如果没有虚线（没有残差），最终只会剩下这个“修改意见” print(x + f) # 19 —— 加上虚线（有残差）以后，“稿子”和“修改意见”都在，19 = 10 + 9
```

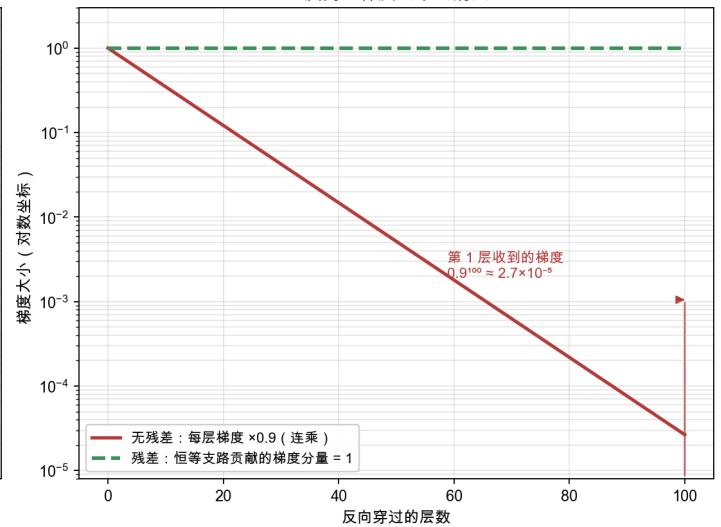
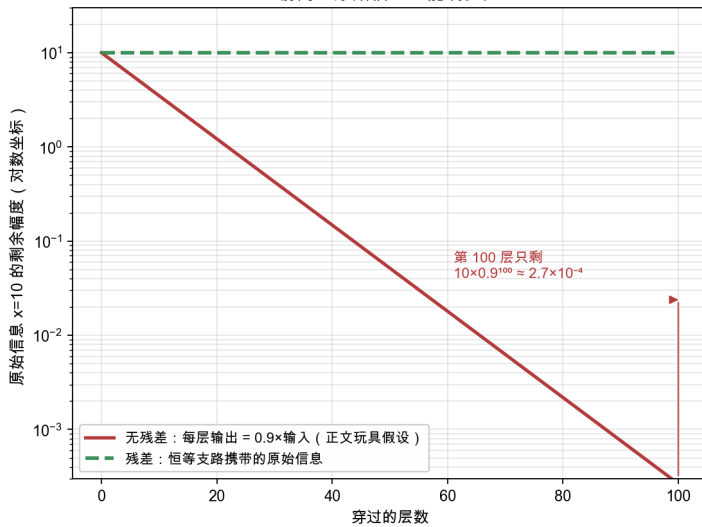
对照一下：f 就是图里 Attention 方框算出来的东西；x + f 这一步，就是图里那个 + 方框在做的事——原始的 10（虚线传过来的）和修改出来的 9（实线传过来的）加在一起，才是最终的 19。没有残差时，Attention 的输出会直接替换掉原来的 x（结果只剩 9，原始的 10 彻底丢了）；有了残差，Attention 只是在原有的 x 上追加一个修改量（结果是 19，10 和 9 都还在）。

把“没有捷径 vs 有捷径”放到 100 层的链条上看（纵轴取对数，才能同时看到两种数量级）：

残差连接：没有捷径时信息与梯度都指数衰减，恒等支路是一条不衰减的高速公路

前向：原始信息还能剩多少

反向：梯度会不会消失



左图（前向）：无残差的链条每层把输入改写成 $0.9 \times$ 输入，原始信息 $x=10$ 传到第 100 层只剩 $10 \times 0.9^{100} \approx 2.7 \times 10^{-4}$ ——这就是“100 个人传话”在数字上的样子；而恒等支路让原始的 10 一路直通，每一层都完整保留（上面 $19 = 10 + 9$ 里的那个 10）。右图（反向）：梯度逐层连乘，无残差时传回第 1 层只剩 $0.9^{100} \approx 2.7 \times 10^{-5}$ ，梯度消失；而 $y = x + F(x)$ 对 x 求导得到 $1 + \partial F / \partial x$ ，其中那个 1 来自恒等支路、不经过任何连乘——这就是梯度的高速公路。生成脚本见 [residual_identity_path.py](#)。

Residual 到底提供了什么？换成前面“稿子 + 修改意见”的说法就很好懂：每一层不再需要自己从头写一份新稿子，只需要在原稿基础上，决定“要不要改、改多少”。如果某一层觉得当前稿子已经不错，不需要大改，它只要让自己的“修改意见”趋近于 0（什么都不改），稿子就会原样传下去——这比“每层都要重新写一份完整稿子才能保持不变”容易得多。写成公式就是：完整输出 $y = x$ （原稿）+ $F(x)$ （这一层的修改意见）， $F(x)$ 只负责学“改多少”，不负责“从零生成”。当然，这只是让训练变容易，并不保证“层数越多就一定没坏处”；网络能不能训好，还要看初始化、归一化、优化器和数据量这些其他因素。

为什么这样更容易训练？拿两个任务对比一下：任务 A 是“让模型学会输出和输入完全一样”；任务 B 是“让模型学会输出 = 输入 + 一点点修改”。任务 A 其实更难——模型要精确地把输入原样照抄一遍才能不出错；任务 B 反而更简单，因为什么都不做（修改意见 = 0）就已经能达到“原样保留”的效果了，模型只需要在真正需要改动的地方，慢慢学出一个不为 0 的修改量。这也是 Residual Learning（残差学习）名字的来源——学的是“改动量”，不是从零学“新稿子”。

补充阅读：残差连接的梯度路径

主线只需要记住： $y = x + F(x)$ 同时保留原输入和变换后的增量。若继续观察反向传播，Loss 对输入的梯度也会分成两条路径：一条经过恒等支路，一条经过 F 。

标量情形可以写为：

$$\frac{\partial \text{Loss}}{\partial x} = \frac{\partial \text{Loss}}{\partial y} \frac{\partial y}{\partial x} = \frac{\partial \text{Loss}}{\partial y} \left(\frac{\partial y}{\partial x} + \frac{\partial F}{\partial x} \right)$$

这个公式的作用：说明残差加法为什么提供了一条不经过 F 内部连乘的梯度路径。

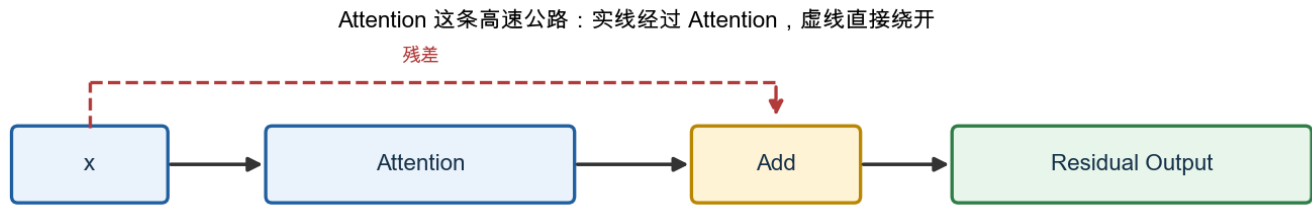
- 第一项：经过 $x \rightarrow y$ 恒等支路的梯度；
- 第二项：经过变换分支 F 的梯度；
- 向量情况下， $\partial F / \partial x$ 应理解为 Jacobian，正文不要求展开。

这条恒等路径通常改善深层网络的梯度传播，但两条梯度还会相加或抵消，归一化和初始化也会影响数值，因此不能概括为“无论多深都能无损传回”。

一些 GPT 实现还会缩小残差输出投影的初始化标准差，例如：

```
std = 0.02 / math.sqrt(2 * n_layer)nn.init.normal_(block.mlp.c_proj.weight, mean=0.0, std=std)
```

这里 `n_layer` 是 Block 数量，缩放用于控制多层残差相加后的激活方差。它属于具体初始化策略，不是理解残差连接前必须掌握的公式。Transformer 每完成一个模块都会执行 $x \rightarrow \text{Attention} \rightarrow + \rightarrow \text{Residual Output}$ ，然后进入 FFN，同样是 $x \rightarrow \text{FFN} \rightarrow + \rightarrow \text{Output}$ 。因此每一个 Block 实际上都有两条高速公路：一条给 Attention，一条给 FFN。



生成脚本见 [residual_highway_diagram.py](#)。

Attention 并不是替代原来的信息，而是在原来的基础上不断补充、不断修正。

本节小结：[OK] 深层网络容易训练困难 [OK] 信息和梯度都会在多层传播中逐渐衰减 [OK] Residual 为信息提供了一条“高速通道” [OK] Transformer 学习的是“增量修改”，而不是完全重写。一句话：保留已有知识，再逐步修正。

四、为什么需要 Layer Normalization（层归一化）？

Residual Connection 让 Transformer 终于能够稳定地训练几十层甚至上百层。但新的问题又出现了。

假设输入 $x=10$ ，经过 Attention 输出 12，经过 FFN 输出 18，再经过 Attention 输出 31，FFN 输出 52……数字越来越大（也可能越来越小，例如 $10 \rightarrow 6 \rightarrow 2 \rightarrow 0.3 \rightarrow 0.01$ ）。那么下一层应该如何学习？

深层网络训练时，不同层的激活尺度可能相差很大；残差不断叠加后，数值也可能逐层放大或缩小。尺度不稳定会让梯度和优化过程更难控制。早期文献常用 Internal Covariate Shift（内部协变量偏移）解释归一化的作用，但今天更稳妥的说法是：归一化约束激活尺度、改善数值条件并稳定梯度，从而让深层网络更容易优化。

一个大胆的想法：每层都重新整理。既然每一层输出的数据范围都不同，为什么不每经过一层都重新整理一下？例如 [103,105,101,104] 在不考虑可学习仿射参数时，标准化后约为 [-0.17,1.18,-1.52,0.51]——均值接近 0、方差接近 1，下一层学习起来会更轻松。

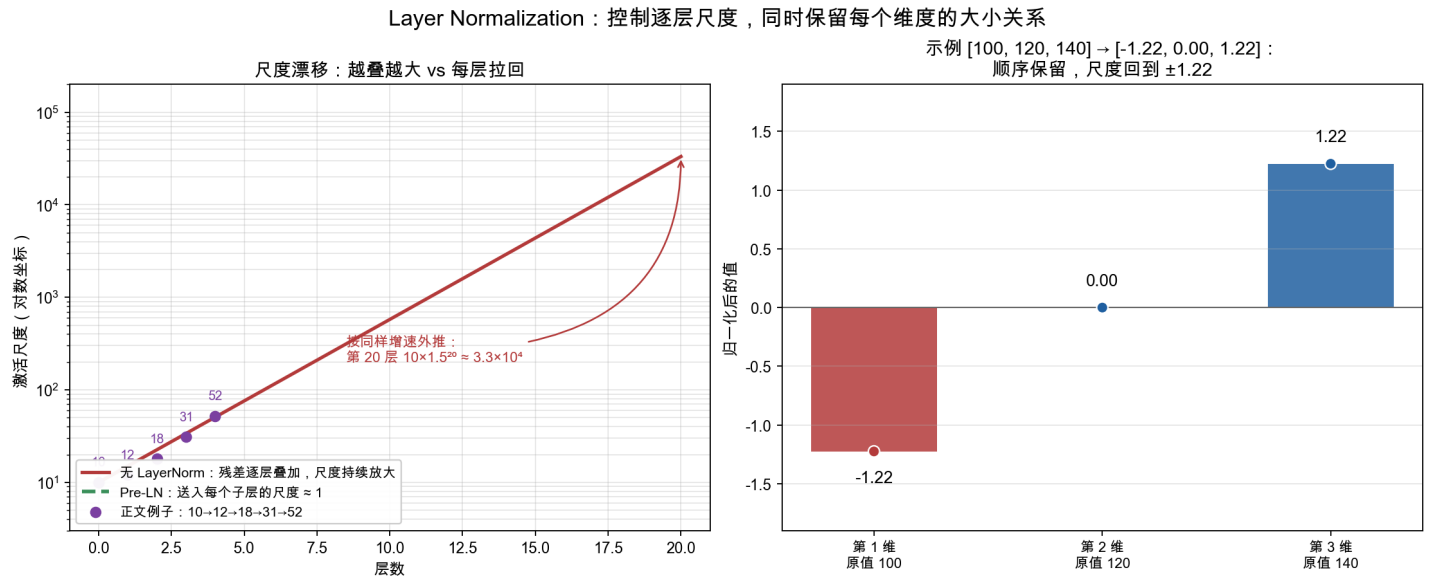
什么是 Normalization？Normalization（归一化）可以理解成：把数据重新调整到一个更稳定、更容易学习的范围。模型并没有丢失“谁最大、谁最小”这些信息，只是把尺度重新统一了。

示例：体验归一化。

```
import torchx = torch.tensor([100.0, 120.0, 140.0])layernorm = torch.nn.LayerNorm(3)print(layernorm(x)) # 例如 tensor([-1.22, 0.00, 1.22])
```

最大的仍然最大，最小的仍然最小，只是数据范围更加稳定。

把“逐层尺度漂移”和“归一化”的效果画在一起：

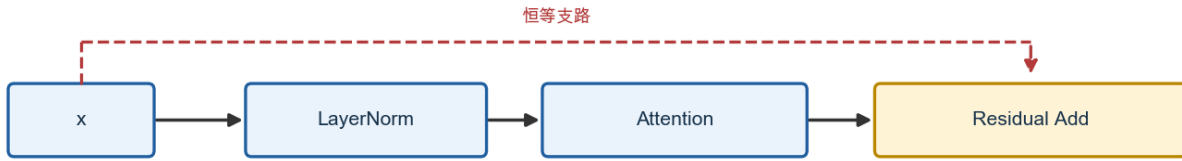


左图：残差不断叠加增量时，激活尺度逐层放大——紫色点是本节开头的 $10 \rightarrow 12 \rightarrow 18 \rightarrow 31 \rightarrow 52$ ，按同样的增速外推到第 20 层就是 $10 \times 1.5^{20} \approx 3.3 \times 10^4$ （注意纵轴是对数坐标）；Pre-LN 结构在每个子层入口做 LayerNorm，把送入 Attention/FFN 的尺度重新拉回 1 附近。右图：本节示例的 [100, 120, 140] 归一化成 [-1.22, 0.00, 1.22]——三个数的大小顺序原样保留，只是尺度从“上百”回到“ ± 1.22 ”。还要说明一点：Pre-LN 归一化的是送入子层的张量，残差主干本身的尺度仍会随深度缓慢增长，这正是第三节补充阅读里 GPT-2 缩小初始化标准差的原因。生成脚本见 [layernorm_scale_control.py](#)。

为什么叫 Layer Normalization？很多人容易把它和 Batch Normalization 混淆。最大区别是：BatchNorm 会比较不同样本（样本 1、样本 2、样本 3 一起计算均值和方差）；而 LayerNorm 只关心当前 Token 自己的向量，只对这一组数字进行归一化，不参考其他 Token，也不参考 Batch 中其他句子。因此 Transformer 才能很好地处理不同长度、不同 Batch，甚至推理阶段只有一个样本的情况。

Residual 和 LayerNorm 为什么总是在一起？Residual 会把旧表示与新增量相加，归一化则帮助控制送入子层或后续层的数值尺度。两者的先后顺序有两种常见结构：原始 Transformer 使用 Post-LN，现代 GPT/LLaMA 更常使用 Pre-LN。本章后续实现采用 Pre-LN：

Pre-LN 结构：先归一化，再计算；虚线恒等支路绕过整个子层



生成脚本见 [preln_residual_diagram.py](#)。

纯标准化阶段会保持同一向量中的线性顺序，但实际 LayerNorm 还有逐维可学习的 γ 、 β ，训练后不应概括为“永远不改变各维度大小关系”。它的主要作用是改善数值条件和优化稳定性，而不是保证任意深度都能稳定训练。

本节小结：[OK] 深层网络各层的激活尺度可能相差很大 [OK] 数值会随残差叠加逐层放大或缩小 [OK] LayerNorm 把每个 Token 的向量归一化到均值 0、方差 1 的稳定范围 [OK] 它只针对当前 Token 自己的向量，不跨 Token、不跨样本 [OK] 现代 Transformer 采用 Pre-LN（先归一化再计算）。一句话：每层都先把尺度整理好，训练才更稳。

五、为什么需要 Dropout？

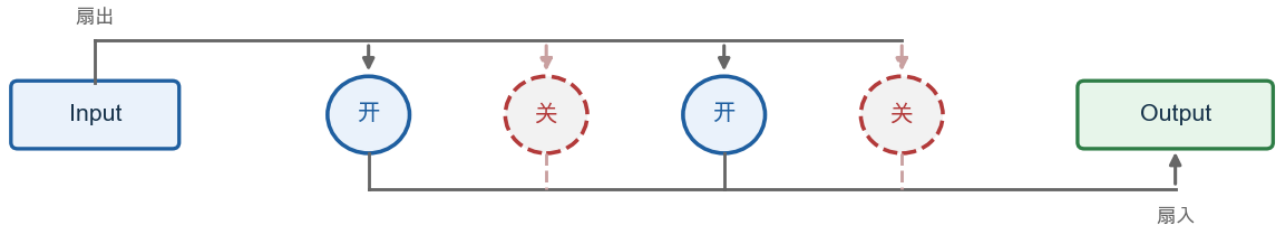
Transformer Block 已经拥有 Multi-Head Attention、FFN、Residual、LayerNorm，模型已经非常完整。但训练时经常出现一个问题：模型在训练集上表现很好，换到没见过的数据上却突然变差。这种现象叫 Overfitting（过拟合）。

一个类比：假设一个学生只背了课本上的 10 道例题，考试时如果原题重现，他能考 100 分；但只要换一道新题，他就完全不会做。这个学生并没有真正理解知识，只是记住了答案。神经网络也会出现同样的问题：如果参数量足够大，模型可能会“记住”训练数据里的每一个细节，而不是学习真正的规律。

为什么会发生过拟合？模型中的某些神经元，可能会过度依赖另一些特定的神经元，形成一种“惯性组合”，只对训练数据有效。例如某三个神经元总是一起激活，只要凑齐这个组合，模型就直接给出训练时见过的答案，而不是重新推理。

一个大胆的想法：训练时故意“捣乱”。既然模型会过度依赖某些固定组合，那么训练过程中随机“关闭”一部分神经元会怎样？

Dropout：随机关闭部分神经元（● 实心=正常工作，○ 虚线=本次被关闭）



生成脚本见 [dropout_neurons_diagram.py](#)。

因为每次训练，关闭的神经元都不一样，模型没有办法始终依赖某一个固定组合，只能学习更通用的规律。这就是 Dropout。

示例：体验 Dropout。

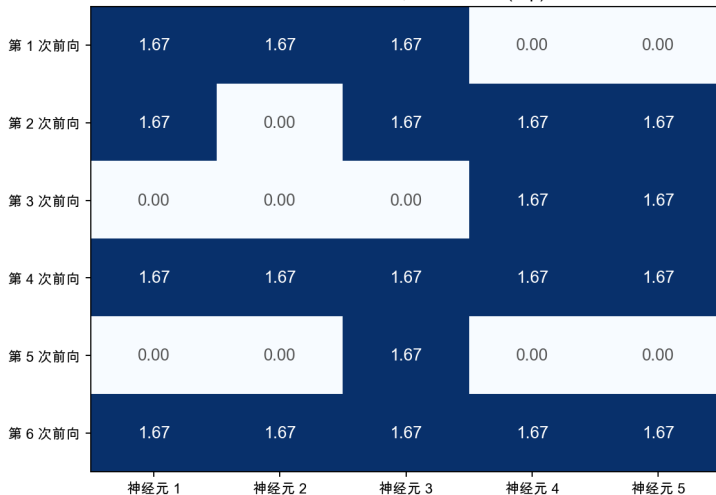
```
import torchx = torch.tensor([1.0, 1.0, 1.0, 1.0, 1.0])dropout = torch.nn.Dropout(p=0.4) # 随机关闭 40% 的神经元print(dropout(x))# 可能输出：tensor([1.67, 0.00, 1.67, 1.67, 0.00])
```

多运行几次，你会发现每次被关闭的位置都不一样。

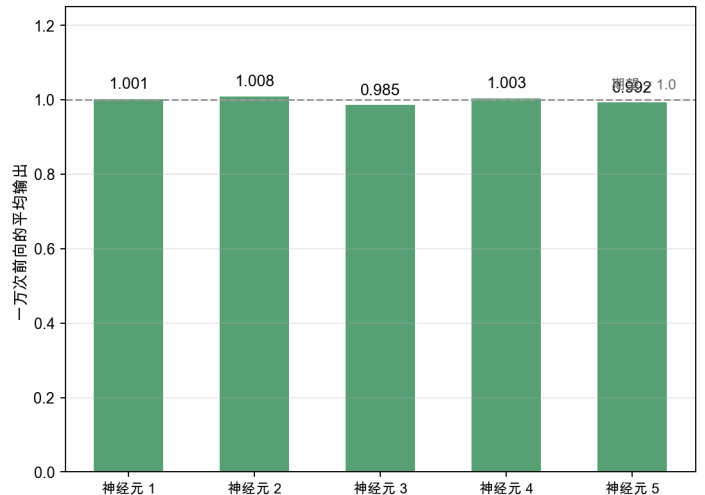
把多次运行的结果摆在一起看：

Dropout：单次前向随机关闭，大量重复后期望保持不变

同一输入 $[1, 1, 1, 1, 1]$ ， $p=0.4$ ：
每次关闭的位置都不同，存活者 = $1/(1-p) \approx 1.67$



存活概率 $0.6 \times$ 放大 $1.67 = 1.00$ ：
缩放保证的是期望不变



左图：同样输入 $[1, 1, 1, 1, 1]$ 、 $p=0.4$ ，六次前向被关闭的位置各不相同——模型没法依赖任何固定组合；每次存活的位置都被放大到 $1/(1-0.4) \approx 1.67$ 。右图：每个位置重复一万次再取平均，都回到 ≈ 1.0 ——缩放保证的是“期望不变”（存活概率 $0.6 \times$ 放大 $1.67 = 1.00$ ），单次前向仍然是“随机制造缺陷”。生成脚本见 [dropout_random_mask.py](#)。

为什么剩下的数字变大了？假设关闭 40% 的神经元，如果不做任何调整，总体信号强度会减少 40%。为了保持训练稳定，PyTorch 会自动放大剩下的神经元（除以 1-p），确保输出总量基本保持不变。这也是为什么示例中 1.0 变成了 1.67（约等于 1/(1-0.4)）。

Dropout 只在训练时使用。推理阶段（Inference）不会使用 Dropout，因为预测时应该使用模型的全部能力，而不是随机关闭。因此几乎所有框架都会区分：

```
model.train() # 训练模式，Dropout 生效model.eval() # 推理模式，Dropout 不生效
```

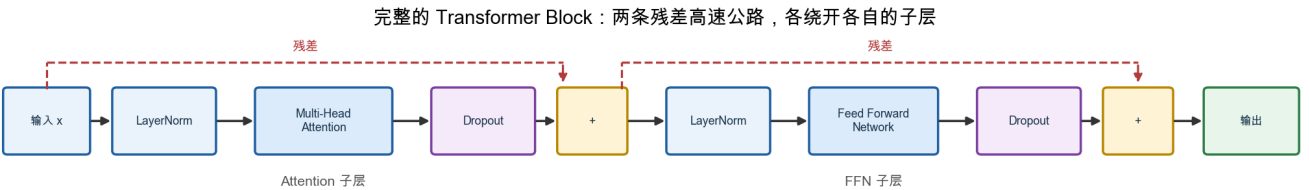
Transformer 几乎每个模块后面都会加一次 Dropout：Attention Weight 计算完成后、FFN 输出后、Embedding 输出后……目的都一样：防止某个模块的输出被过度依赖。

本节小结：[OK] 过拟合：模型记住答案，而不是学会规律 [OK] Dropout 会在训练时随机关闭部分神经元 [OK] 目的是防止模型依赖固定的神经元组合 [OK] Dropout 只在训练阶段生效，推理阶段会关闭。一句话：Dropout 通过“随机制造缺陷”，逼迫模型学习更鲁棒、更通用的规律。

六、一个完整的 Transformer Block

前面几节，我们已经分别学习了 Multi-Head Attention（信息交流）、Feed Forward Network（独立思考）、Residual Connection（保留原始信息）、Layer Normalization（稳定训练）、Dropout（防止过拟合）。本节，我们将把它们正式组装成一个完整的 Transformer Block。

完整结构：



生成脚本见 [transformer_block_full_diagram.py](#)。

整个 Block 可以概括为两句话：

```
x = x + Dropout(Attention(LayerNorm(x)))x = x + Dropout(FFN(LayerNorm(x)))
```

为什么 LayerNorm 放在前面？这种写法叫 Pre-LayerNorm（先归一化，再计算），是现代 GPT、LLaMA 普遍采用的方式，相比早期 Transformer 论文中的 Post-LayerNorm（先计算，再归一化）训练更加稳定，尤其是在层数很深的时候。本书不深入比较两者差异，只需记住：现代大模型基本都采用 Pre-LayerNorm。

理论上，一个 Pre-LayerNorm Block 的前向过程就是：

```
attention_output = Attention(LayerNorm(x))x = x + Dropout(attention_output)ffn_output = FFN(LayerNorm(x))x = x + Dropout(ffn_output)
```

标准 Transformer Block 保持输入输出 Shape (batch, seq_len, d_model) 不变，因此多个 Block 可以顺序堆叠。实际可堆叠的深度受到显存、计算量、训练稳定性和数据规模约束。完整的 PyTorch 类定义、Causal Mask 和训练循环统一放在 11.1 节实践中，避免在理论章和实践章重复实现同一个类。

七、进入 Mini GPT 实践前：位置信息

Self-Attention 根据 Token 内容计算关联，公式本身不含位置编号，因此模型无法仅凭 Attention 判断一个 Token 原来排在第几位。但语言顺序会改变含义——“我喜欢你”和“你喜欢我”用的字完全相同，意思却完全相反。所以完整 GPT 必须额外注入位置信息。11.1 节先采用最简单的可学习绝对位置向量：

```
模型输入 = Token Embedding + Position Embedding
```

位置 0,1,2,... 各自对应一个可训练向量，与同位置的 Token Embedding 相加。为什么 Attention 缺少位置信息、还有哪些方案、现代模型为什么改用 RoPE，由第 12 章完整展开，本章只先把它接进代码。

八、进入 Mini GPT 实践前：Causal Mask

第 10 章为了讲清 Attention 的计算过程，一直默认每个 Token 都能查看整个输入序列。但 GPT 做 Next Token Prediction 时，位置 t 只能使用 t 及其左侧的 Token；如果训练时看到右侧未来 Token，就等于提前看到答案。

Causal Mask（因果掩码）会把 Attention Score 矩阵中“看向未来”的位置屏蔽掉：

```
Key 位置  0  1  2  3Query 0  允许 × × ×Query 1  允许 允许 × ×Query 2  允许 允许 允许 ×Query 3  允许 允许 允许 允许
```

实现时通常在 Softmax 之前，把被屏蔽位置的 Score 设为 $-\infty$ ；经过 Softmax 后，这些位置的权重变成 0。这样，训练阶段可以并行计算整段序列，同时保持“只能根据过去预测未来”的因果约束。11.1 节会把这个 Mask 接进 MultiheadAttention；第 14 章讨论 Transformer 家族时再比较 Encoder 的双向 Attention 与 Decoder 的 Causal Attention。

九、堆叠多个 Block

一个 Transformer Block 只完成一次“交流 + 思考”。GPT-3 堆叠了 96 层，LLaMA-2 70B 堆叠了 80 层。每多堆叠一层，模型就能学习更深层次的语言规律：浅层可能学习词法、语法，深层可能学习逻辑推理、常识、上下文关联。



生成脚本见 [block_stack_diagram.py](#)。

不过，这里拼出来的 Transformer Block，用的还是最"朴素"的版本：位置信息用的还是可学习的绝对位置向量（第 12 章的 RoPE 还没有讲）、LayerNorm 而不是更现代的 RMSNorm、FFN 用的是 GELU 而不是现在更主流的 SwiGLU，Multi-Head Attention 里每个 Head 也都有自己独立的 K、V。下一章，我们就来看看：从 2017 年的原始 Transformer 到今天的 GPT、LLaMA、Qwen，这个 Block 到底经历了哪些改造。

十、本章总结

Transformer Block 完整清单：

组件	作用
Multi-Head Attention	让每个 Token 从多个角度收集上下文信息
Residual Connection	保留原始信息，避免深层网络退化
Layer Normalization	稳定每一层的数据分布
Feed Forward Network	对每个 Token 独立进行非线性加工
Dropout	防止模型过度依赖固定神经元组合

至此，我们已经完整拼出了现代 Transformer 最核心、最基础的构建单元——Transformer Block。GPT、BERT、LLaMA、Qwen，几乎所有现代大语言模型，本质上都是由大量这样的 Block 堆叠而成。

不过，本章目前只完成了"能跑通 forward"，还从来没有真正训练过、也没有生成过一个字。下一节，我们将用这个 TransformerBlock 从零训练一个真正能生成文字的 Mini GPT：[11.1 从零训练 Mini GPT](#)。

参考资料

- Andrej Karpathy 《Neural Networks: Zero to Hero》系列（micrograd / makemore / Let's build GPT / nanoGPT）：<https://www.youtube.com/@AndrejKarpathy>
- 李宏毅 《机器学习》（Self-Attention / Transformer 经典入门讲解）：<https://space.bilibili.com/3546866876680925>
- Sebastian Raschka 《Build a Large Language Model (From Scratch)》配套代码与全书资源：<https://github.com/rasbt/LLMs-from-scratch>